

Teatro View Engine – Feature Overview and Refactoring Plan

Current Features of Teatro (Top-Down Overview)

Declarative View DSL & Rendering: Teatro provides a Swift DSL for building UI-like “views” in a platform-neutral way. Core protocols like `Renderable` allow composing text and layout elements (e.g. `Text`, `VStack`, `HStack`) and rendering them to outputs ¹ ². A result-builder `@ViewBuilder` API lets developers declaratively construct hierarchies of views (similar to SwiftUI) which are then converted into a **layout tree** for rendering ³. For example, a `Stage` container with a title can group scenes or content; it will render as a labeled section in output ⁴ ⁵. Teatro’s layout engine can overlay views (`ZStack`) and apply basic styling (bold, italic, underline) via `TextStyle` enums ⁶ ⁷. All views ultimately produce text-based output by default (monospaced for alignment), which can be transformed into various formats by plugins.

Screenplay (Fountain) Format Support: Teatro can parse and render film script documents written in the Fountain markup format. It includes a full `FountainParser` that implements the Fountain spec (scene headings, character cues, dialogue, action, transitions, notes, etc.) without regex, following a documented state-machine plan ⁸. Parsed Fountain elements are mapped to an enum `FountainElement` (e.g. `.sceneHeading`, `.characterCue`, `.dialogue`, etc.), each with a custom render style (scene headings become Markdown headers, character cues uppercased, transitions suffixed with “>>”, etc.) ⁹ ¹⁰. A `FountainSceneView` is provided to take raw `.fountain` text and produce a rendered view of the screenplay structure ¹¹ ¹². This allows GPT or other agents to generate screenplay snippets and immediately see them formatted as a script in the output ¹³. For example, a Fountain scene with headings and dialogue will render with scene headers prefixed by `#` and dialogue indented under character names ¹⁴ ¹⁵. The parser supports customization via rule sets (e.g. adding `LOC` as a scene heading prefix, toggling notes) ¹⁶.

Storyboard & Animation System: Teatro includes an **animation timeline DSL** to script sequences of visual “frames” (states) and transitions. Developers can define a `Storyboard` with named `Scene{}` blocks (each containing a view tree for that state) and transitional steps like `Transition(style: .crossfade, frames: 10)` to animate between scenes ¹⁷ ¹⁸. The DSL supports crossfades and tweened transitions with a given frame count and optional easing. At runtime, a storyboard can be expanded into discrete frames via `storyboard.frames()`, which produces an array of renderable frames across all scenes and transitions ¹⁹ ²⁰. These frames can be used for **deterministic animation rendering**. For example, Teatro can output a series of PNG images for each frame (via an `Animator.renderFrames` utility) or compile all frames into a single SVG with embedded `<animate>` directives for a scalable animated graphic ²¹ ²². This enables use cases like stepwise visualization of a narrative or musical timeline. Indeed, use cases include *“frame-by-frame reflections or GPT planning steps, MIDI timeline playback, and screenplay line reveals”* ²³ – all scenarios where an AI or logic produces a sequence of states that need rendering. Teatro’s `TeatroPlayerView` (see below) can also play these frames in real time.

MIDI 2.0 Integration and Sequencing: A distinctive feature of Teatro is its deep integration with **MIDI 2.0** for timing and synchronization. It includes a declarative Swift DSL for MIDI sequences: the

`MIDISequence` result-builder allows listing `MIDI2Note` events with properties like channel, note number, 32-bit velocity and duration ²⁴ ²⁵. This makes it easy for AI agents to compose melodies or rhythms in code. For example, one can write:

```
let melody = MIDISequence {  
  MIDI2Note(channel: 0, note: 60, velocity: 0.8, duration: 0.5)  
  MIDI2Note(channel: 0, note: 64, velocity: 0.8, duration: 0.5)  
  MIDI2Note(channel: 0, note: 67, velocity: 0.8, duration: 1.0)  
}
```

to define a three-note melody ²⁶. Each `MIDI2Note` can be encoded to raw Universal MIDI Packet words via `UMPEncoder` for output or storage ²⁷ ²⁸. Crucially, **MIDI drives Teatro's animations**: the duration of each note in a sequence can determine how long a corresponding frame remains on screen ²⁹ ³⁰. The `TeatroPlayerView` runtime uses a `MIDISequence` to time scene playback – e.g. a melody's note lengths syncing with frame durations – ensuring that visuals advance on beat with the music ³⁰. This tight coupling of MIDI timeline and rendering timeline is a core design of Teatro. (MIDI 1.0 is supported as a legacy fallback; Teatro can translate between new and old MIDI: e.g. `MIDI1Bridge.umpToMIDI1()` for UMP → MIDI 1.0 bytes and vice versa ³¹ ³².)

Audio Playback (TeatroSampler): To actually *hear* the MIDI sequences, Teatro includes a cross-platform **sampler engine**. The **TeatroSampler** is an actor-based component that routes MIDI 2.0 note events to a selected backend synthesizer, either **FluidSynth** (SoundFont-based) or **Csound** (orchestra-based) ³³. Internally, it wraps C libraries (`libfluidsynth` and `libcsound`) to generate sound. The sampler ensures audio and visual timelines stay in sync (e.g. no drift between what's being displayed and what's played) ³⁴ ³⁵. It exposes a simple API: one can instantiate `TeatroSampler(backend: .fluidsynth(sf2Path: "soundfont.sf2"))` or `.csound(orchestra: "synth.orc")`, then call `await sampler.trigger(MIDI2Note(...))` to play a note ³⁶. Under the hood, the FluidSynth backend loads the SF2 sound font and opens an audio driver (CoreAudio on macOS, PulseAudio on Linux) to output sound, and it translates each note-on/off into libfluidsynth calls ³⁷ ³⁸. The Csound backend compiles and starts a Csound orchestra (with -odac for audio-out) and sends note events via score messages (`csoundInputMessage`) with frequency and amplitude calculated from the MIDI note and velocity ³⁹ ⁴⁰. Both backends run on background tasks (FluidSynth spawns an audio thread via its driver; Csound is ticked via `csoundPerformKsmps` loop in a Task) to produce continuous sound ⁴¹ ⁴². This audio engine allows Teatro to function as a self-contained “stage” where an AI can not only script visuals but also play music in real time. Notably, **TeatroPlay** is a command-line tool that uses this capability to pipe MIDI data to the system's synthesizer (FluidSynth on Linux or Apple's built-in sampler) for quick audio playback ³².

Output Formats & Rendering Backends: Teatro can output content in multiple formats thanks to a plugin-driven **renderer registry**. By default, it supports: plain text/Markdown, HTML, SVG (static and animated), and images. The rendering system works by taking the final `Renderable` view (or array of frames) and feeding it to a chosen **RendererPlugin**. For example, the built-in `SVGRenderer` wraps each line of the view's monospaced text output into `<text>` elements at incremental Y positions, inside an `<svg>` canvas ⁴³ ⁴⁴. The `HTMLRenderer` similarly wraps output in a `<pre>` tag within a basic HTML page for quick viewing in browsers ⁴⁵. An `ImageRenderer` uses Cairo (if available) to draw text onto a PNG image surface for raster output ⁴⁶ ⁴⁷. For animated content, `SVGAnimator.renderAnimatedSVG()` takes a `Storyboard` and produces a single SVG with `<animate>` or `<animateTransform>` elements encoding the frame transitions (e.g. fading out one scene's group over N seconds) ⁴⁸ ⁴⁹. The CLI can invoke this via `--format svgAnimated` to get a

timeline animation in one file ⁵⁰. Additionally, a `MarkdownRenderer` can emit the output as a fenced code block (useful for embedding in docs or logs) ⁵¹. There is even a `CodexPreviewer` which outputs a commented block containing the view's type and rendered content, intended for feeding back into AI models for reflection ⁵² ⁵³. All these renderers register themselves with a global `RendererRegistry` at startup ⁵⁴. The CLI and APIs can query available formats by identifier (e.g. "svg", "png", "codex") and route the output accordingly ⁵⁵ ⁵⁶. This plugin architecture also allows third parties to add custom renderers; an extension mechanism exists whereby an external package can call `RendererRegistry.register(MyRenderer.self)` to make new `--format` options available ⁵⁷ ⁵⁸.

CLI Tools and Workflow: Teatro ships with several command-line utilities for headless operation and scripting. The primary one is `RenderCLI`, which can take various input files and produce the chosen output format. It accepts `.fountain` scripts, `.storyboard` files (the textual storyboard DSL), `.ump` (MIDI UMP sequence dumps), `.ly` (LilyPond files), `.csd` (Csound scores), `.session` logs, etc., and determines an appropriate view to render ⁵⁹ ⁶⁰. For instance, giving a `.fountain` file to `RenderCLI` will use `FountainSceneView` internally and default to an SVG or text output; a `.storyboard` plus `--format svgAnimated` will produce an animated SVG of the storyboard timeline ⁵⁰. `RenderCLI` can "watch" a file with `-w` to auto-reload on changes (useful for live preview during editing) ⁶¹ ⁶². It also has flags to bridge or replay MIDI: e.g. `--midi1-bridge` will convert a UMP input file to MIDI 1.0 bytes (or vice versa) using the built-in bridge and either save to file or pipe to stdout ⁶³ ⁶⁴. There's even an `--watch-rtpmidi` mode that listens for live UMP messages over the network (RTP-MIDI) and can save or forward them – effectively treating Teatro as a monitoring tool for incoming MIDI streams ⁶⁵ ⁶⁶. Another tool, `teatro-play`, is a simple player that reads UMP or MIDI data (from file or stdin) and plays it through the sampler with a given `SoundFont` (`--sink fluidsynth --sf2 <file>`) ³², providing a quick way to audition generated music. `FountainCLI` is an experimental CLI target that ties together Fountain scripts with the preview system; it may allow rendering a screenplay with an interactive timeline (it depends on `TeatroRenderAPI`, `TeatroSDLBackend`, etc. to utilize the new GUI features) ⁶⁷. Finally, `TeatroSamplerDemo` is a small executable that demonstrates the sampler (loading the bundled `assets/sine.orc` and `example.sf2` and playing some notes), mainly for testing the audio setup ⁶⁸ ⁶⁹.

LilyPond Musical Notation Export: As a niche feature, Teatro allows generating sheet music via **LilyPond**. It includes a `LilyScore` renderable that holds LilyPond `.ly` notation text ⁷⁰. When rendered normally, it just outputs the LilyPond source text. But it also provides `renderToPDF()`, which will write the `.ly` content to a temp file and invoke the `lilypond` command-line tool to compile it into a PDF (and PS) file ⁷¹ ⁷². This lets GPT agents create a snippet of LilyPond music notation and have Teatro produce a print-ready score. For example, given a LilyPond string defining a C-major scale, calling `score.renderToPDF("theme")` will produce `theme.pdf` with standard music notation ⁷³ ⁷⁴. (Note: LilyPond must be installed on the system, and Teatro simply calls out to it; it's not a built-in renderer.) The `LilyScore` is compatible with Teatro's view system – it conforms to `Renderable` – so it can be embedded in a Stage or used with the Animator to, say, include a musical score image in an animated sequence ⁷⁵. This feature aims to let AI plan compositions and immediately see the musical notation output, bridging textual music description and actual sheet music.

Server-Sent Events (SSE) over MIDI 2.0: Teatro can act as a bridge for **streaming text events** (such as live tokens from an AI) carried within MIDI 2.0 streams. In practice, this means that an HTTP SSE stream (from a language model, for example) can be encapsulated into special MIDI 2.0 messages (UMP **Flex Data** packets for small chunks and **SysEx8** for larger chunks) and fed into Teatro ⁷⁶ ⁷⁷. Teatro's SSE

decoder (`FountainSSEDispatcher`) will extract these JSON-structured “envelopes” and reassemble them into the original text events. Each SSE message is timestamped (using MIDI2’s JR Timestamp) to keep it in sync with the musical beat or timeline ⁷⁷. There is a **reliability layer** implemented: Teatro negotiates a lightweight reliability protocol using MIDI-CI (Capability Inquiry) messages to acknowledge received envelopes and request retransmission of any missed packets ⁷⁸ ⁷⁹. A jitter buffer (~3ms by default on LAN) ensures that minor network jitter doesn’t throw off the timing. In short, Teatro can robustly ingest a live token stream from an AI *in time with music*. The **interactive preview GUI** (see next) even shows live token text and stream stats: it displays overlays of SSE packet groups, round-trip times, and loss/retransmit counts in real time ⁷⁹ ⁸⁰. A sample capture of such a stream is provided (`assets/sse-demo.ump`) for testing ⁸¹. This capability is quite cutting-edge – it effectively uses MIDI 2.0 as a transport for synchronizing AI-driven narrative or lyric content with musical sequences.

Interactive Preview GUI (SDL3 + SwiftUI): In its latest evolution, Teatro is experimenting with an interactive GUI for live previews. The goal is to have a cross-platform **SDL3-powered window** that can render the Teatro output in real time with controls for playback, scrubbing, and interactive visualization of the timeline ⁸² ⁸³. As of now, this feature is **in progress**: the repository contains a `TeatroSDLBackend` module and a `MIDIIntegration` module which together are intended to drive an SDL rendering loop with MIDI 2.0 input. The current implementation is stubbed (no actual SDL graphics yet) to keep things deterministic during testing ⁸⁴. However, the architecture is laid out: a `TeatroPreviewController` API exists to launch a preview window, and an example `TeatroPreviewDemo` executable can run a demo script in preview mode ⁸⁴ ⁸⁵. In practice, when fully realized, this will allow users (or other programs) to call `TeatroRenderer.renderScript` or similar and not only get an SVG snapshot, but also pop up a window where the scene is animating in sync with sound. The preview will visualize MIDI 2.0 events (for example, highlighting beats or keys) and SSE tokens overlaying the script or storyboard timeline ⁸⁶ ⁸⁰. It will also provide interactive controls: e.g. play/pause buttons, the ability to scrub through the timeline or click on a script line to cue the music, etc. The **TeatroPlayerView** mentioned earlier plays a similar role on macOS within SwiftUI: it can display an SVG or sequence of frames in a SwiftUI view (used for Xcode previews or embedding in a Mac app) ⁸⁷ ⁸⁸. Currently the SwiftUI `TeatroPlayerView` is very minimal (just shows an SVG as an image or counts bytes) ⁸⁹ ⁹⁰, but the plan is for it to interface with the core player/engine. The key point is that Teatro is moving towards *interactive*, not just headless, usage. An agent (like FountainAI) could generate a script with cues and, via Teatro’s preview, **immediately see a window with the scene playing out with audio**, enabling rapid creative iteration. This “live stage” aspect is the future of Teatro’s design.

Architecture and Implementation Status

Monolithic Core with Feature Modules: Internally, Teatro’s codebase currently behaves like a **single engine with many subsystems**. The main Swift package target `Teatro` comprises the bulk of the code (the view DSL, renderers, Fountain parser, storyboard system, SSE logic, etc.), and it integrates directly with low-level C libraries for audio ⁹¹. This target links against `CCsound` and `CFluidSynth` wrappers (for the sampler) and pulls in the `MIDI2` package (providing UMP and MIDI-CI types) and SwiftCBOR (likely for MIDI-CI data or other encoding) ⁹¹. The `Teatro` module’s sources span multiple concerns (text rendering, parsing, I/O) in one place. However, to manage new functionality, a few **auxiliary targets** have been introduced: for example, `TeatroSDLBackend` is a separate target that will contain SDL3 window and event loop code (currently mostly stubbed out) ⁹², and `MIDIIntegration` is another target that depends on `Teatro` and `MIDI2` and provides glue logic specifically for the interactive preview (like adapting the MIDI 2.0 streams to the GUI) ⁹³. These are isolated so that the core engine (`Teatro`) remains usable in headless or pure library scenarios without forcing SDL dependencies. Similarly, smaller executable targets exist for each CLI tool (each with its own

`main.swift`). All of these are assembled in the single `teatro` Swift Package (with multiple products) as of now ⁹⁴ ⁹⁵.

Design Patterns and Data Flow: The engine heavily uses **value types and pure functions** for rendering. Views conform to protocols like `Renderable` or `Layouting` and typically implement a `layout()` method that produces a tree of `LayoutNode` structs (representing structured text and spacing) ¹ ⁹⁶. A default protocol extension provides a `render()` method that traverses this layout tree and converts it to plain text (with markdown-style markers for bold/italic) ⁹⁷ ⁹⁸. This text represents the “formatted output” in a device-independent way (monospaced spacing etc.). Renderer plugins then post-process that text to their respective formats (embedding in SVG, HTML, etc.). This pipeline makes it easy to add new output formats, but it also means the GUI effort needs to reverse-map text to graphics. The new SDL backend will likely bypass the intermediate text rendering for on-screen drawing, or use the layout info to draw directly.

Teatro also embraces **Swift concurrency** for certain parts: The sampler is implemented as an `actor` (`TeatroSampler`), isolating the C sound library calls on a single thread and using async message passing for note events ⁹⁹ ¹⁰⁰. SSE streaming uses an actor (`StreamPreviewController`) as well to handle incoming packets, assemble messages, and maintain the token buffer and reliability state without data races ¹⁰¹ ¹⁰². The use of Swift actors and `async/await` throughout ensures thread safety (e.g. the GUI can ingest MIDI on a background task and update the token list safely). The package is built with strict concurrency checks enabled (see Swift settings like `-Xfrontend -strict-concurrency=complete` in `Package.swift`) ¹⁰³, indicating the developer’s intent to catch data races and enforce `Sendable` where possible.

Integration Boundaries: Because the project covers a lot of domains (text layout, music, networking, graphics), certain boundaries are drawn in code organization. Notably, **parsing and domain logic** (Fountain parsing, Storyboard parsing) are kept in pure Swift, while **system-dependent functionality** (audio output, GUI) is funneled through narrow C interfaces. For example, the Csound and FluidSynth integrations are each wrapped by minimal C API bridges (see `Sources/CCsound/csound.h` with just the needed functions declared ¹⁰⁴ ¹⁰⁵). The Swift `FluidSynthSampler` then directly calls those C functions (like `new_fluid_synth`, `fluid_synth_noteon`) encapsulated in an actor context ¹⁰⁶ ¹⁰⁷. This keeps the unsafe calls contained. Similarly, the planned SDL integration will likely use SDL’s C APIs via a bridging header. At the higher level, Teatro tries to maintain deterministic behavior for most of the engine (so it can run in a server or test context). For example, the current “interactive” preview code does not actually open a window unless explicitly run in the demo – instead it’s structured to allow headless testing with stub outputs ⁸⁴. The **public API surface** has been relatively small and focused: a new `TeatroRenderAPI` module was introduced as a stable facade for external use, offering simple functions like `TeatroRenderer.renderScript(...) -> RenderResult` instead of exposing all internals ¹⁰⁸ ¹⁰⁹. This decouples external apps (like the FountainAI app or codex-deployer) from the internal complexities. Internally, `TeatroRenderer.renderScript` calls into `FountainParser` and the SVG renderer as we saw ¹¹⁰ ¹¹¹. Other `renderXYZ` functions assemble storyboards or session logs and delegate to the appropriate renderer or encoder ¹¹² ¹¹³.

Documentation and Tests: The repository is accompanied by extensive documentation chapters (as cited throughout), design proposals, and a task matrix. This indicates the current architecture is the result of iterative proposals (e.g. a blueprint for the Render API, a proposal for MIDI 2.0 Sampler integration, etc.). There are targeted test suites: e.g. `StoryboardDSLTests`, `MIDITests`, `RendererFileTests`, etc., validating pieces like SSE envelope parsing and rendering outputs. Continuous integration runs SwiftLint and all tests on macOS and Linux to ensure cross-platform

consistency ¹¹⁴. This heavy emphasis on docs and testing is great, but it also reflects the engine's **complexity** – new contributors face a learning curve to absorb all these moving parts.

In summary, **Teatro's implementation** is powerful but **somewhat monolithic**, with a core engine that does “everything” (parsing, layout, MIDI handling, etc.) and a few bolt-on modules for new features. The code is highly concurrent and modular internally (via protocols and result builders), but at the package level it isn't yet fully separated by concern. All features ultimately compile into one library product (`Teatro`) aside from the small preview and CLI modules ⁹¹ ¹¹⁵. This monolithic structure means that to use one aspect (say the Fountain renderer), one currently pulls in the entire engine including Csound, FluidSynth, and MIDI logic. It also makes the onboarding challenging – the next section addresses these concerns.

Critique of the Current Architecture

While Teatro's feature set is rich and innovative, the **current architecture feels like a collage of capabilities** without clear separation. A new developer diving in must contend with *multiple domains intertwined* in one codebase: screenplay formatting, graphics rendering, MIDI encoding, audio synthesis, streaming protocols – all living under the `Teatro` module. This monolithic design hinders **maintainability and onboarding** in several ways:

- **Separation of Concerns:** The boundaries between distinct functionalities are blurry. For instance, the core `Teatro` target knows about both rendering logic and low-level audio drivers. There isn't a clean delineation like “UI/Rendering vs Audio vs MIDI” – everything is interdependent inside one module. This can confuse newcomers (e.g., you find MIDI parsing code alongside text layout code) and makes it harder to modify one subsystem in isolation. The introduction of `TeatroRenderAPI` was a step toward isolating public rendering APIs, but internally the module still depends on almost all engine details. As a result, even a simple usage (like rendering a Fountain script to SVG) currently pulls in the MIDI2 library, Csound, FluidSynth, etc., which is unnecessary coupling.
- **Complexity and Onboarding Difficulty:** Because the repo has grown feature-by-feature, it now reads like an *amalgam of experimental features*. A developer trying to understand “how Teatro works” must parse a variety of concepts (some quite advanced, like SSE streaming reliability or MIDI-CI) that might not be needed for basic use. The structure does not guide the reader through a logical progression; instead one finds, say, an `Audio/` folder and a `MIDI/` folder and a `ViewCore/` folder all at the top level. There is an **ONBOARDING.md absence** – unlike FountainKit, which after refactoring has a clear AGENTS guide and modular packages, Teatro lacks a high-level guide to its structure, making it hard to know where to start. The feeling of “collage” stems from the codebase trying to do many things at once without a unifying abstraction for all of them.
- **Redundancy and Inconsistency:** Due to iterative development, there are some redundant layers and naming confusions. For example, the presence of both a `TeatroRenderAPI` and a `RenderAPI` target in Package.swift is not immediately obvious – it appears one is meant as the stable interface and the other perhaps as internal extensions (the latter includes the `StreamPreviewController`, etc.). This duality could confuse contributors about which API to use or extend. Similarly, features like SSE streaming and MIDI-CI exist in the code, but how they integrate isn't clearly delineated at the module level. One might wonder: is SSE streaming part of the core rendering engine or an add-on? Currently, it's embedded in core (as FountainSSE*

classes in `Sources/MIDI/SSE` within the main target), which is questionable design-wise. Ideally such specialized functionality would be in an optional module.

- **Scaling and Maintainability Issues:** As Teatro moves toward a real GUI (SDL window, interactive controls), the current architecture will face challenges. The rendering pipeline that works for static SVG/PNG might not be best for real-time drawing. The team will likely introduce new types (for example, textures or font metrics for SDL). If those are just jammed into the main module, complexity spikes. Also, performance tuning becomes tricky when everything runs on the main thread – the current design leans on generating text and letting C libraries handle background work (audio, etc.), but as we add a 60 FPS GUI loop, clear separation of the GUI thread vs logic thread will be needed. Right now, adding that within the monolith could risk race conditions or slowdowns if not carefully refactored.
- **Dependency bloat and Reuse:** Because the monolith includes heavy dependencies (FluidSynth ~25MB library, Csound, etc.), any project that wants just a slice of Teatro's functionality must absorb all that. This is suboptimal. For example, FountainKit (the organization's main app code) might only need Teatro's rendering of SVG and the MIDI timeline, but not the audio playback. In the current state, it has to include everything. This also means build times and binary sizes are larger than necessary. If Teatro were refactored into smaller components, it would be possible to reuse just the needed parts in other contexts (e.g. a lightweight "TeatroCore" without audio for a web backend vs a full "TeatroPlayer" for desktop). As it stands, the tight coupling means **reusability is limited**.
- **Documentation vs Reality:** The documentation paints a picture of a more modular system than actually exists. For instance, the docs refer to "Teatro stays a rendering/runtime engine" and not building product-specific UIs ¹¹⁶, implying a clean interface. Yet internally, product-specific logic (like Codex integration hooks, or CLI specifics) can be found in core (e.g., the `CodexPreviewer` is in the rendering backend code). The AGENTS.md for GUI integration spells out policies like *"Encapsulate SDL3 usage behind internal APIs; do not expose SDL types publicly"* ¹¹⁷ – a good principle, but currently SDL is barely integrated so it's more aspirational. The presence of a "task-matrix.md" suggests some tasks might be incomplete or in progress. In short, there's a bit of a **gap between the planned architecture and the implemented one**, likely due to the project evolving rapidly. This can lead to confusion or duplicated efforts (some features in docs might not be fully functional yet, etc.).
- **Monolith History:** We saw a similar situation in the FountainKit repository before its refactoring – it started as a monolithic codebase and was later split into modular packages. FountainKit's refactor greatly improved clarity, grouping core vs services vs UI vs examples separately ¹¹⁸ ¹¹⁹. Teatro would benefit from the same treatment. The lack of module boundaries in Teatro is an architectural debt that now needs addressing.

In summary, **Teatro's architecture needs restructuring** to align with its broad feature set. The engine should be disentangled into logical units so that contributors (or automated agents) can focus on one piece at a time. This will reduce cognitive load, prevent accidental cross-impacts (e.g. a change in audio code breaking the rendering output), and pave the way for the **SDL GUI future** by having a well-defined place for those components. The next section lays out a plan to achieve this, using lessons learned from the FountainKit refactoring and the stated direction of making Teatro a robust SDL-backed engine.

Comprehensive Refactoring Plan

To resolve the issues above, we propose a **modular redesign** of the Teatro repository, turning the “collage” into a structured engine. The guiding idea is to separate major concerns into distinct Swift packages or targets (much like FountainKit’s Packages) while preserving the current functionality. We imagine Teatro’s future as an **SDL-backed rendering engine** with clearly defined subcomponents. Refactoring will involve reorganizing code, clarifying APIs, and establishing new module boundaries. Here is the plan:

1. Define Clear Modules (Swift Packages) for Teatro

We will split the repository into a few focused modules, each with a clear responsibility. For example:

- **TeatroCore – Core rendering engine** (non-GUI). This will include the declarative view DSL, Fountain parsing and formatting, Storyboard/Animation system, MIDI 2.0 DSL and basic timeline handling, and all the renderer plugins for outputs (SVG, text, etc.). TeatroCore should have *no direct dependency on system libraries* like SDL, FluidSynth, or Csound – it represents the pure logic that can run anywhere (including Linux servers or in tests). It would depend on the external `MIDI2` package for UMP and MIDI-CI structures, and on SwiftPM packages for things like `SwiftArgumentParser` if needed for core logic (though CLI will be separate). By isolating core, we ensure that the fundamental “render this script to SVG” functionality is lightweight and portable.
- **TeatroAudio** (or **TeatroSamplerKit**) – **Audio output subsystem**. This will contain the Csound and FluidSynth integrations and the `TeatroSampler` actor. Its job is to produce sound for MIDI events. By splitting this off, someone who doesn’t need built-in audio can exclude this module. TeatroAudio will depend on TeatroCore (for the `MIDI2Note` types, etc.) but not vice versa. It will encompass `CCsound`, `CFluidSynth`, `FluidSynthSampler`, `CsoundSampler`, and the `SampleSource` protocol definitions. We might also include the MIDI 1.0 bridging here or in core (depending on usage) – bridging could remain in core if it’s pure logic, but any *live playback* pieces would be here.
- **TeatroGUI – SDL3-based interactive GUI**. This module will house the SDL window management, rendering loop, and input handling. Essentially, everything in `TeatroSDLBackend` and the related preview integration would go here (we may rename it from “Backend” to something like “TeatroGUI” for clarity). It will depend on TeatroCore (to render frames, etc.) and on TeatroAudio (if we want the GUI to also control audio – e.g., toggling mute or selecting output device). The GUI module will expose a high-level API like `TeatroPreviewController` or `TeatroPlayer` that the main app can use to launch a preview window, start/stop playback, etc. Importantly, this module will encapsulate all SDL-specific code behind Swift APIs, as per project policy ¹¹⁷. It means new types like `SDLWindow`, `SDLRendererContext` etc., and utility to map Teatro’s layout/text to actual drawing calls (perhaps using `SDL_ttf` for text rendering). By having a dedicated module, platform-specific tweaks (like different behavior on macOS vs Linux) can be handled internally without cluttering the core logic.
- **TeatroCLI – Command-line interface tools**. We will create a separate package for executables (akin to FountainApps in FountainKit). This can include the `RenderCLI` target, `teatro-play`, and any other user-facing CLI. These will depend on TeatroCore and optionally TeatroAudio or TeatroGUI depending on what they do. For instance, `RenderCLI` (which outputs files) might only depend on Core (and on `ArgumentParser`), whereas a potential `teatro-preview` CLI

could depend on GUI module to open a window. Keeping CLI separate ensures that core libraries don't import `ArgumentParser` or assume file I/O – they remain library-pure.

- **Telemetry/SSE module (optional)** – Given the complexity of **SSE-over-MIDI** and how it might be reused in other contexts (`FountainTelemetryKit` in `FountainKit` covers similar ground ¹²⁰), we should consider isolating the **streaming telemetry** parts. This could be a `TeatroTelemetry` module that contains `FountainSSEEnvelope`, `FountainSSEDispatcher`, `FountainSSEReliability`, and possibly the `StreamPreviewController`. Its role would be to handle streaming events and reliability, independent of the UI. `TeatroGUI` would depend on `TeatroTelemetry` to display overlays, and `TeatroCore` might also depend on it if core needs to parse SSE logs or embed SSE in outputs. Alternatively, if we decide SSE streaming is always tied to the GUI, we could keep it in `TeatroGUI` – but carving it out as `Telemetry` has the benefit that other projects (like `FountainKit`'s gateway) could use it without dragging in the whole GUI. This aligns with `FountainKit`'s approach of having **`FountainTelemetryKit for MIDI2 and SSE`** ¹²⁰. We will likely model `Teatro`'s SSE handling on that separation.
- **TeatroDocs** (optional) – Not a code module, but we should organize documentation (maybe under `Docs/` or a GitHub Pages site) to reflect the new structure. An **`ONBOARDING.md`** or updated `README` will be important to explain the modules and how to get started.

These modules can be structured as local Swift packages under a `Packages/` directory (monorepo style). For instance, we would have `Packages/TeatroCore`, `Packages/TeatroAudio`, etc., each with its own `Package.swift` defining its products. The top-level `Package.swift` can then aggregate them or we use SwiftPM workspaces. This is exactly how `FountainKit` was reorganized – into packages like `FountainCore`, `FountainTelemetryKit`, `FountainApps`, etc ¹¹⁸. Adopting that here will make `Teatro` **modular and scalable**.

2. Maintain a Cohesive Public API

After refactoring, from a user's perspective, using `Teatro` should still be straightforward. We likely want to keep **`TeatroRenderAPI`** as a facade that reexports or wraps the core functions. For example, `TeatroRenderer.renderScript` should remain available (perhaps as part of `TeatroCore` or as its own target that depends on `Core` and `GUI` if needed). We might merge the current `RenderAPI` target into this as well (to eliminate confusion).

Concretely, we might provide a **unified import** (e.g. `import Teatro` or `import TeatroKit`) that pulls in common functionality. SwiftPM allows one product to reexport others. For instance, we could have an umbrella product `Teatro` that depends on `TeatroCore` + `TeatroAudio` + `TeatroGUI` for those who want “everything”, and also publish more granular products (like `TeatroCore` alone for headless usage on Linux). The **`AGENTS.md`** in the `FountainAI` examples shows usage of `TeatroRenderAPI` from the package ¹²¹; we will ensure that continues to work with minimal changes. The refactor should not break external code relying on these APIs (except for requiring maybe an extra import if we split modules – but we can mitigate that by reexporting symbols).

3. Implement in Phases

Refactoring a live codebase demands care to not disrupt functionality. We plan these steps:

- **Phase 0: Baseline and Tests.** First, ensure we have a solid test baseline (all current tests passing on CI). Add additional tests around integration points if needed (to catch regressions when

moving code). E.g., test that rendering a fountain script to SVG with a certain input produces expected output (snapshot tests) ¹²², test that TeatroSampler can play a note (maybe in a dummy sound mode), etc. These tests will guide the refactor.

- **Phase 1: Create module skeletons.** Set up the `Packages/` directory structure with new `Package.swift` files for each planned module. For example, `Packages/TeatroCore/Package.swift` defining a library product `TeatroCore` with no dependencies yet; `Packages/TeatroAudio` with dep on `TeatroCore`; and so on. We will initially *duplicate* or reference the existing code in these modules (without removing from old structure) to get the build system hooked up. This might mean temporarily two copies or using SwiftPM's ability to have targets depend on sources in another subpath. We prefer an incremental approach: e.g., move one category of files at a time from the monolith target into the new target and adjust references.
- **Phase 2: Migrate code to TeatroCore.** Begin moving purely logical components into the `TeatroCore` module. This includes the **View DSL and rendering logic**: files like `Text.swift`, `VStack.swift`, `LayoutNode.swift`, `FountainParser.swift`, `Storyboard.swift`, `SVGRenderer.swift`, etc., go into `TeatroCore`. After moving, update their imports (they may now need `import Foundation` or other basics, but should not import missing symbols from other parts). Resolve any dependency issues: e.g., if during this move we find code that calls Csound functions (which belong in audio), we isolate that call to later move. At the end of this step, `TeatroCore` should compile independently and pass its unit tests (we'd move the corresponding tests to `Packages/TeatroCore/Tests`).
- **Phase 3: Migrate Audio to TeatroAudio.** Take all audio-related code (Csound and FluidSynth integration, `TeatroSampler.swift`, `Midi1Bridge.swift` if applicable) and move to the `TeatroAudio` module. This module will depend on `TeatroCore` (for the `MIDI2Note` types, and perhaps for logging utilities or any core models). After moving, ensure that `TeatroAudio`'s tests (e.g. `SamplerTests`) pass when run in isolation. The C targets `CCsound` and `CFluidSynth` will be targets in this package (or separate packages if needed). Make sure linking works on both macOS and Linux (setting up `pkg-config` or linking flags for `fluidSynth`, etc., likely similar to current Setup). At this point, `TeatroCore` + `TeatroAudio` combined should provide all functionality the old monolith did, but without GUI.
- **Phase 4: Migrate CLI and others.** Shift the CLI targets (`RenderCLI`, `TeatroPlay`, etc.) into a **TeatroCLI package**. This package will depend on `TeatroCore` (for rendering) and `TeatroAudio` (for the play command). We will update the `ArgumentParser` commands to import the new modules. One benefit: we can remove the odd exclusion of CLI sources from the main target ¹²³ because now CLI is separate. After this, running `swift run RenderCLI` from root should still work, just pulling from the new locations.
- **Phase 5: Integrate TeatroGUI (SDL3).** This step likely overlaps with feature development. We will take the existing `Sources/TeatroSDLBackend` and `Sources/MIDIIntegration` code and move it into a new **TeatroGUI** (or `TeatroPreview`) package. This module will depend on `TeatroCore` (and `TeatroAudio` if audio control is needed in GUI). We'll refactor how it interacts: e.g., the `TeatroPreviewController` might become a class in this module that internally uses SDL. We will connect the `StreamPreviewController` (from telemetry) here to feed SSE tokens into GUI overlays. Notably, we should implement the real SDL event loop now (using SDL3). This includes initializing SDL, creating a Window and Renderer, and drawing text or simple shapes to represent the frames. Given that full graphical fidelity might be complex, we can start

with a basic approach: use an SDL texture to render the SVG output (perhaps by rasterizing it) or draw text via `SDL_ttf` for each frame line. Over time, this can be improved to draw directly. The key is that all SDL-specific calls reside in this module. We also ensure **no SDL leaks into TeatroCore's API** ¹¹⁷. If needed, TeatroCore could offer an abstraction for a frame (like a list of rendered lines with positions) that GUI consumes to draw. We will also incorporate cross-platform concerns (like ensure it runs headlessly in CI by using dummy video driver or skipping if no display, etc.). By the end of this, `swift run TeatroPreviewDemo` should open an actual window on supported platforms and remain headless (no-op) on CI.

- **Phase 6: SSE Telemetry (if separate module):** Move SSE handling to `TeatroTelemetry` module and integrate with Core/GUI. This involves taking `FountainSSEEnvelope` and related classes out of core and into Telemetry. TeatroCore would then not know about SSE specifics; instead, TeatroTelemetry would provide parsed token streams that GUI can subscribe to. If Telemetry module is too small, we may decide to keep it within Core but as a clearly separated namespace. We'll evaluate based on reuse needs.
- **Phase 7: Cleanup and Unify APIs.** Once everything is in place, we drop the old monolithic target. The top-level `Package.swift` will now reference the sub-packages. We ensure that external usage is smooth: likely we provide a top-level library product "TeatroKit" or simply "Teatro" that depends on all core modules for convenience. The `AGENTS.md` for FountainKit refactor emphasized avoiding cross-package testable imports and duplications ¹²⁴ ¹²⁵ – we will follow those guidelines (e.g. if something is used by both Core and Audio, put it in Core rather than duplicating). We'll remove any leftover stub or duplicate code. For example, if both TeatroCore and TeatroGUI had a `TeatroPlayerView`, we unify them (perhaps one is for SwiftUI, one for SDL – we might keep SwiftUI preview extension in Core or GUI as an add-on). We also clarify naming: if `RenderAPI` target is redundant, we remove it, consolidating its `StreamPreviewController` into Telemetry or GUI.
- **Phase 8: Documentation and Guides.** Update **README and docs** to reflect new module names and usage patterns. Provide a high-level **ONBOARDING.md or AGENTS.md** that explains the new structure (similar to FountainKit's agent guide ¹²⁶). This will list the packages and their responsibilities, coding standards (e.g. "TeatroCore should remain platform-agnostic", "TeatroAudio must clean up audio drivers on deinit", etc.), and workflow for contributing. We will mention that to get all features you use `Teatro` umbrella, but you can mix and match (for example, use TeatroCore alone for generating SVG on a server).
- **Phase 9: Testing & CI adjustments.** Ensure that running tests for each package individually works (e.g. `swift test --package-path Packages/TeatroCore`). Adjust CI to run all package tests. Also test installation as Swift Package from another project – e.g. the `HelloFountainAITeatro` example should still compile when pointing to the refactored repo (we might need to adjust its dependency product names).

Throughout these phases, we'll use version control to keep the history clear (this likely will span multiple commits or pull requests, which can reference this plan as a roadmap). We should also ensure **backward compatibility** or at least clear migration notes. Since the project is pre-1.0, we can bump the version (e.g. 0.2.0) for the new structure, and in release notes state which imports changed.

4. Leverage FountainKit Refactoring Experience

The FountainKit refactor set a precedent on how to modularize effectively. We will mirror that approach. For instance, FountainKit created a `FountainCore` for base models and a `FountainTelemetryKit` for MIDI/SSE concerns ¹²⁷. We plan similarly with TeatroCore and a possible Telemetry module. FountainKit also had *strict dependency rules* (libraries shouldn't import executables, etc.) ¹²⁴ – we'll enforce the same. One concrete example: FountainKit's telemetry package contains MIDI 2.0 streaming and SSE features reused by multiple modules ¹²⁰. In our case, rather than having separate MIDI logic in Teatro and FountainKit, post-refactor we could even consider unifying – perhaps FountainKit could use Teatro's MIDI/SSE modules instead of its own, reducing duplication. Our refactoring will make such collaboration possible by isolating those features.

We will refer to FountainKit's git history and AGENTS.md as a guide for structure and even wording in docs. For instance, we might follow a similar format when writing Teatro's agent guide (listing each package and its purpose, coding standards, etc., akin to the excerpt from FountainKit's AGENTS ¹¹⁸). This not only gives consistency across the org's projects but also signals to contributors that Teatro is now as well-organized as FountainKit.

5. Incorporate SDL Backend Fully (Future-proofing)

Since a major direction is “Teatro's future as an SDL backend”, our refactoring sets the stage for that by isolating the GUI. Once refactored, we will proceed to implement missing pieces of the SDL integration as outlined in the existing AGENTS spec ⁸³ ⁸⁶. The architecture objectives in that spec (e.g. unified GUI backend, interactive rendering loop at ~60 FPS, MIDI 2.0 data pipeline for live visualization, etc.) will be realized within the TeatroGUI module. The refactoring itself helps by giving that work a proper home, preventing it from cluttering core logic or breaking headless usage. We will use feature flags or build flags such that, for example, if SDL is not available (building on a platform or CI without it), we can compile TeatroCore and TeatroAudio alone and skip GUI.

Additionally, by modularizing, we could even provide an **option to not compile the GUI** (for users who only want CLI or server-side usage). Similarly, one could exclude TeatroAudio if they only need the render outputs. This flexibility serves both development (faster iteration on one module) and deployment (smaller footprint for specific use cases).

6. Ensure Outcomes and Benefits

The success of this refactor will be measured by several outcomes of interest:

- **Improved Onboarding:** New developers (or AI agents contributing code) should be able to read the TeatroCore module in isolation to grasp the rendering basics, without immediately being overwhelmed by audio or streaming code. Each module will have its own README documenting its API and relationship to others. The top-level AGENTS.md will provide the map. This modular clarity lowers the entry barrier.
- **Encapsulation and Maintainability:** Each module will enforce boundaries. For instance, TeatroCore will be rigorously platform-agnostic and deterministic (no global state beyond perhaps a `RendererRegistry`). TeatroAudio will manage lifecycle of sound drivers so that issues like memory leaks or hangs are isolated. Bugs will be easier to locate – e.g. if a MIDI parsing issue occurs, it's in Core; if a sound glitch, it's in Audio. The codeowners or maintainers can also be split by expertise.

- **Reusability:** If FountainAI or others only need the rendering, they can depend only on `TeatroCore` (and maybe `TeatroRenderAPI` facade) to generate outputs ¹⁰⁸. This avoids dragging in SDL or heavy libs. Conversely, if someone wanted to embed the interactive player in a macOS app, they can depend on `TeatroGUI` and get everything (with its dependencies transitively). We'll thus support multiple usage scenarios elegantly.
- **Alignment with Vision:** With a modular structure, we can more rapidly iterate on the SDL GUI (which is central to Teatro's envisioned future) without risking regressions in the core rendering outputs. The GUI team can work in `TeatroGUI` module, possibly even versioning it separately if needed. The core engine can continue to be used in deterministic testing and in cloud environments (where obviously an SDL window wouldn't be used). This dual modality is crucial for FountainAI's needs (headless vs interactive usage). After refactoring, *Teatro will "feel" like a cohesive engine composed of well-defined parts, rather than a grab-bag of features*. This sets the stage for it to become the exclusive rendering backend for the broader Fountain system (no more ad-hoc renderers elsewhere, as per the blueprint goals ¹²⁸ ¹²⁹).

Finally, we will formalize this plan and its execution steps in an actionable **AGENTS.md** within the Teatro repo (see below), similar to how it was done for FountainKit. That document will serve as both a roadmap for implementation and a guide for contributors once the refactor is complete.

Actionable Refactoring Guide (AGENTS.md for Teatro)

```
# Teatro Refactor - Modular Architecture Initiative

- **Owner:** Contexter@Fountain-Coach
- **Version:** 0.1 (2025-09-27)
- **Scope:** Restructure the Teatro repository from a single monolithic package into a set of modular Swift packages. Establish clear boundaries for core rendering, audio output, GUI preview, and CLI tools, to improve maintainability and onboarding. Align Teatro's codebase with its future role as the SDL-based rendering backend for FountainAI and related projects.

- **ID:** teatro-modular-refactor
- **Name:** Teatro Modular Architecture & SDL Integration Readiness

## 1. Description

This refactoring breaks Teatro's "collage" of features into well-defined modules:

- **TeatroCore** - The core deterministic rendering engine (view DSL, Fountain parser, timeline animation, MIDI encoding, file format outputs). No GUI or audio side-effects.
- **TeatroAudio** - MIDI 2.0 native sampler and audio backends (FluidSynth, Csound) packaged behind a clean async API.
- **TeatroGUI** - Interactive GUI preview using SDL3 for rendering and input, plus MIDI2 real-time visualization.
- **TeatroCLI** - Command-line tools and executables for using Teatro in scripts and pipelines.
- **TeatroTelemetry** (optional) - Streaming SSE over MIDI2 handling and diagnostics (if not within Core).
```

Each module will live in ``Packages/`` with its own tests and documentation. External usage of Teatro is preserved via a unified API module (``TeatroRenderAPI``) that taps into these internals.

By doing this, we **decouple** components and prevent unwieldy growth. New contributors can focus on one module at a time. The SDL GUI can evolve separately, and headless usage remains lightweight.

This refactor also mirrors the successful modularization of FountainKit ¹¹⁸, ensuring consistency across the organization.

2. Architectural Objectives

2.1 Modular Boundaries & Dependencies

Define strict boundaries:

- TeatroCore should not depend on any C libraries or SDL - it's pure Swift (plus ``MIDI2`` and utility packages).
- TeatroAudio depends on Core (for data types) but Core *does not* depend on Audio.
- TeatroGUI depends on Core (for content) and optionally Audio (for controlling playback), but core logic doesn't depend on GUI.
- CLI depends on whatever modules needed but no module depends on CLI.
- Telemetry (if separate) depends on Core and MIDI2, and GUI depends on Telemetry for overlay data.

This ensures, for example, we can build Core on a platform without audio or GUI, and still run all its features.

2.2 Public API Stability

Continue to provide a simple public interface for common use-cases. The ``TeatroRenderAPI`` (or just ``Teatro`` umbrella) will re-export core functions like ``renderScript``, types like ``TeatroPlayerView``, etc., so external code sees minimal changes. We avoid breaking existing integrations like HelloFountainAITeatro ¹³⁰ - they might just need to update import names. Mark new module boundaries with proper Swift package products and use semantic versioning to signal changes.

2.3 SDL3 Integration (Future-proof)

Lay the groundwork for fully integrating SDL3:

- The TeatroGUI module will contain all needed SDL initialization and event loop code. We will ensure it can create a window, draw text or simple shapes for frames, and handle user input (keyboard/mouse) to control playback.
- Keep GUI rendering logic abstracted: possibly implement a basic text-based rendering first (to mirror current SVG output) and plan to expand with more graphical fidelity (fonts, colors, etc.).
- Ensure GUI runs on macOS and Linux reliably (use feature flags or dummy devices in CI to run tests).

2.4 Maintain or Improve Performance

Splitting into modules should not degrade performance. In fact, it may

improve build times (parallel builds of modules) and allow more focused optimization. We will monitor that rendering an SVG or playing audio has no regression. The GUI loop will target 60 FPS for modest content, as per requirements ¹³¹.

2.5 Documentation & Onboarding

For each module, create a README or doc explaining its usage and APIs. Update the top-level documentation to include a map of the new structure (which part does what, with links to internal docs). Include an **ONBOARDING** section so that a newcomer knows to start by reading TeatroCore's README (for example) before diving into code.

3. Deliverables (Planned Modules & Components)

- **TeatroCore** - Core library
 - **Content**: View DSL (`Renderable`, `ViewBuilder`, `Text`, `VStack`, etc.), layout engine (`LayoutNode`), FountainParser & FountainElement model, Storyboard DSL + Animator, MIDI DSL (MIDI2Note, MIDISequence, UMP encoding), Renderers (SVGRenderer, HTMLRenderer, MarkdownRenderer, etc.), MIDI1Bridge, and other pure logic.
 - **Tests**: Fountain parsing tests, rendering snapshot tests (SVG output), MIDI encoding tests, etc.
 - **Product**: `TeatroCore` (library)
- **TeatroAudio** - Audio backend library
 - **Content**: `CCsound` and `CFluidSynth` C targets, `CsoundSampler` actor, `FluidSynthSampler` actor, `TeatroSampler` facade, SampleSource protocol, plus any audio-related utilities.
 - **Tests**: SamplerDemo or dedicated tests that initialize the sampler (perhaps using the dummy sine.orc and a small SF2) and ensure note triggers work.
 - **Product**: `TeatroAudio` (library)
- **TeatroGUI** - GUI/Preview library
 - **Content**: SDL integration (window creation, rendering context), an event loop that coordinates with TeatroCore (to get frames) and TeatroAudio (for sound). `TeatroPreviewController` class or actor to manage a running preview. SwiftUI `TeatroPlayerView` can live here (behind `#if canImport(SwiftUI)` flag) or remain in Core if it only uses Core. SSE overlay UI components (if any, e.g., drawing token text on screen).
 - **Tests**: Integration tests if possible (e.g., run the preview in a headless mode for a short duration and verify that no errors occur, or use Xvfb in CI for SDL on Linux ¹³² ¹³³). Otherwise, reliance on manual testing for GUI aspects with automated unit tests for non-UI logic (like the data flow).
 - **Product**: `TeatroGUI` (library)
- **TeatroCLI** - Executables
 - **Content**: Sources for `RenderCLI` (the ArgumentParser definitions and command implementations), `teatro-play`, `TeatroPreviewDemo` (which will now use TeatroGUI API to launch a preview). Possibly `FountainCLI` if it remains

(or that may fold into RenderCLI as a subcommand).

- ***Note:** We might not need a separate package for CLI; we could keep CLI targets in the main repo Package.swift but segregated from libraries. However, a separate package ``TeatroCLI`` helps enforce that libraries don't depend on CLI.

- ***Product:** Executables: ``render-cli``, ``teatro-play``, ``teatro-preview`` (if introduced), etc.

- ****Packages/TeatroTelemetry/**** - (Optional) Streaming & Reliability

- ***Content:** SSE envelope structures, dispatcher (FlexData/SysEx8 parsing), reliability algorithm (ack tracking, sequence gap detection), and the ``StreamPreviewController`` that bridges UMP streams to high-level token events.

- ***Notes:** This could also be part of TeatroCore if we consider it fundamental. But isolating it is cleaner if we foresee using it in FountainKit's telemetry. If separate, it produces a ``TeatroTelemetry`` library that both FountainKit and TeatroGUI can use.

- ***Tests:** Already there are tests like FountainSSEEnvelopeTests, etc. Those move here. Ensure end-to-end test with a sample UMP stream to tokens.

- ****Top-level integration:****

- Update the root ``Package.swift`` to either use these as local package dependencies or simply remove in favor of a workspace (in which case clients would use ``.package(path: "Packages/TeatroCore")``, etc., or we publish these to git). Perhaps simpler: keep one Package.swift that defines all targets but group them logically (similar result, but using SwiftPM target dependencies rather than separate packages). Decision to be finalized, but outcome is the same modular separation.

- ****Docs & Guides:****

- ``AGENTS.md`` (this document) updated upon completion with any changes in plan/version.

- ``ONBOARDING.md`` or contributing guide explaining module structure.

- Update README usage examples (if any new imports or initialization steps).

- Possibly a migration note: "If you previously used ``import TeatroRenderAPI``, continue to do so; internal modules are mostly invisible except if you import individual ones for advanced use."

4. Implementation Plan (Milestones & Tasks)

- ****Milestone 1:**** Create module scaffolds and move View/Core code

- [] Set up ``Packages/TeatroCore`` with a minimal Package.swift and move core source files.

- [] Adjust import paths in moved files, ensure ``swift test`` passes for TeatroCore alone.

- [] Remove those files from the old target, run overall tests.

- ****Milestone 2:**** Move Audio code to TeatroAudio

- [] Create ``Packages/TeatroAudio``, move Csound and FluidSynth wrappers and Swift sampler code.

- [] Fix any cross-module references (Core types used in Audio, e.g., convert MIDI2Note to simpler form if needed or make MIDI2Note type public in Core).
- [] Test that `TeatroSamplerDemo` still runs using new module (adjust its target dependencies).
- **Milestone 3:** Establish TeatroCLI package
 - [] Move RenderCLI source into `Packages/TeatroCLI` (or keep in main but retarget dependencies to new modules).
 - [] Update CLI commands to use new APIs (e.g., if `TeatroCore` is separate, ensure `import TeatroCore` or via umbrella).
 - [] Test CLI commands: e.g., `swift run render-cli input.fountain --format svg` yields identical output as before.
- **Milestone 4:** Integrate Telemetry (if separate)
 - [] Move SSE parsing and StreamPreviewController to `Packages/TeatroTelemetry`.
 - [] Ensure TeatroGUI (or Core) uses this for SSE features (e.g., GUI calls `StreamPreviewController.ingestFlex()` on incoming MIDI bytes).
 - [] Run SSE tests to confirm no breakage.
- **Milestone 5:** Activate SDL in TeatroGUI
 - [] Implement basic SDL window open/close in TeatroGUI (perhaps a `TeatroWindowManager` class).
 - [] Tie a render callback to TeatroCore: e.g., for each frame tick (maybe 60Hz or driven by MIDI events), get current frame's text layout from core and blit to window.
 - [] Add user controls: e.g., Esc to close, Space to pause, etc., as a proof of concept.
 - [] Test `TeatroPreviewDemo` manually on macOS and Linux.
- **Milestone 6:** Finalize API and remove old structure
 - [] Eliminate the old `Sources/Teatro` monolithic target from Package.swift, instead have the new packages produce the needed products.
 - [] Ensure the umbrella `TeatroRenderAPI` either becomes part of TeatroCore or is a small target depending on Core+GUI to expose combined functionality (decide whether to keep the name or just have users import TeatroCore + TeatroGUI explicitly).
 - [] Bump package version, update documentation references from old paths to new.

Each milestone will be a pull request (if using PRs) or at least a distinct commit, to allow bisecting if needed.

5. Testing & Quality

- **Continuous Integration:** Update CI to build and test all modules. For example, add jobs for `swift test -c debug Packages/TeatroCore`, etc., in addition to the root build. Use an Xvfb headless display for SDL tests on Linux CI ¹³⁴ to run GUI tests without a physical GPU.
- **Unit & Integration Tests:** All existing tests must pass. Additional

tests will be added for new module boundaries (e.g., a test that instantiating TeatroSampler in isolation doesn't crash, a test that RenderCLI's format registry lists expected formats from Core's plugins).

- **Performance Checks:** Render a known complex Fountain script to SVG before and after refactor to ensure rendering time is not worse. Do similarly for a MIDI playback.
- **Documentation Checks:** Ensure that any code examples in Docs/Chapters are updated to new module imports if needed (e.g., if now one must `import TeatroCore` or if `TeatroPlayerView` moved). Possibly provide an alias in code for backward compat (like `public typealias TeatroPlayerView = TeatroGUI.TeatroPlayerView` if needed).

6. Impact and Rollout

- **Impact:** No runtime behavior change intended (except the new actual GUI features coming online). The main impact is on developers: a more organized codebase.
- **Migration for users:** Minimal. If using SPM, they might see new targets. We'll maintain `package.name = "teatro"` so existing `.package(url: "... Teatro.git", branch: "main")` still works. If we change product names, we'll document it. Internally, FountainAI's next update can adopt the new imports (which should be trivial).
- **Timeline:** The refactor can be done incrementally over a couple of weeks, with the core split happening first (which yields immediate benefits). The SDL integration may continue beyond the refactor as its own project, but this structure allows working on it concurrently without disturbing core.

7. Conclusion

By modularizing Teatro, we **future-proof** its development and make it contributor-friendly. The engine's innovative features - from screenplay rendering to synchronized music visualization - will be much easier to extend and maintain in this new architecture. This refactoring brings Teatro in line with the architectural rigor of FountainKit ¹¹⁸, ensuring that the Teatro engine can reliably serve as the unified rendering backend (with SDL3 support) for all FountainAI applications going forward.

1 3 6 7 96 97 98 01_CoreProtocols.md

https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Docs/Chapters/01_CoreProtocols.md

2 4 5 02_ViewTypes.md

https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Docs/Chapters/02_ViewTypes.md

8 9 10 11 12 13 14 15 16 08_FountainScreenplayEngine.md

https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Docs/Chapters/08_FountainScreenplayEngine.md

17 18 19 20 **10_StoryboardDSL.md**

https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Docs/Chapters/10_StoryboardDSL.md

21 22 23 30 34 **05_AnimationSystem.md**

https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Docs/Chapters/05_AnimationSystem.md

24 25 26 27 28 29 **07_MIDI20DSL.md**

https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Docs/Chapters/07_MIDI20DSL.md

31 32 84 85 114 **README.md**

<https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/README.md>

33 35 36 **12_TeatroSampler.md**

https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Docs/Chapters/12_TeatroSampler.md

37 38 41 106 107 **FluidSynthSampler.swift**

<https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Sources/Audio/Samplers/FluidSynthSampler.swift>

39 40 42 **CsoundSampler.swift**

<https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Sources/Audio/Samplers/CsoundSampler.swift>

43 44 45 46 47 48 49 50 51 52 53 57 58 **03_RenderingBackends.md**

https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Docs/Chapters/03_RenderingBackends.md

54 **RendererRegistry.swift**

<https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Sources/Plugin/RendererRegistry.swift>

55 56 59 60 61 62 63 64 65 66 **RenderCLI.swift**

<https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Sources/CLI/RenderCLI.swift>

67 68 69 91 92 93 94 95 103 115 123 **Package.swift**

<https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Package.swift>

70 71 72 73 74 75 **06_LilyPondMusicRendering.md**

https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Docs/Chapters/06_LilyPondMusicRendering.md

76 77 78 79 80 81 **12_SSE_Over_MIDI2.md**

https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Docs/Chapters/12_SSE_Over_MIDI2.md

82 83 86 117 131 132 133 134 **AGENTS.md**

<https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/AGENTS.md>

87 88 108 109 **RenderAPI.md**

<https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Docs/RenderAPI.md>

89 90 116 122 128 129 **TEATRO_GUI_RenderAPI_Blueprint_PR_Checklist.md**

https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/TEATRO_GUI_RenderAPI_Blueprint_PR_Checklist.md

99 100 **TeatroSampler.swift**

<https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Sources/Audio/TeatroSampler.swift>

101 102 **StreamPreviewController.swift**

<https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Sources/RenderAPI/StreamPreviewController.swift>

104 105 **csound.h**

<https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Sources/CCsound/csound.h>

110 111 112 113 **RenderAPI.swift**

<https://github.com/Fountain-Coach/Teatro/blob/00e94640d8141e2555c9f01750dbf7c5a65b4ce9/Sources/TeatroRenderAPI/RenderAPI.swift>

118 119 120 124 125 126 127 **AGENTS.md**

<https://github.com/Fountain-Coach/FountainKit/blob/147fa21c71cc301b7342a6bfc321806a30ac413c/AGENTS.md>

121 **Package.swift**

<https://github.com/Fountain-Coach/the-fountainai/blob/7fbe124489fd955846a18fa00ca8d2b78d11df76/Examples/HelloFountainAITeatro/Package.swift>

130 **HelloFountainAITeatroApp.swift**

<https://github.com/Fountain-Coach/the-fountainai/blob/7fbe124489fd955846a18fa00ca8d2b78d11df76/Examples/HelloFountainAITeatro/macOS/HelloFountainAITeatroApp.swift>